Algebraic Mapping Operators for Knowledge Graph Construction

Sitt Min Oo [a,*], Ben De Meester [a], Ruben Taelman [a] and Pieter Colpaert [a]
[a] *Department of Engineering and Architecture, University of Ghent – imec, Ghent, Belgium*
E-mails: x.sittminoo@ugent.be, ben.demeester@ugent.be, ruben.taelman@ugent.be, pieter.colpaert@ugent.be

Abstract. Declarative knowledge graph construction proliferated in multiple mapping languages, mapping language versions, and mapping engines. It advanced to the point where more studies focus on optimizing the knowledge graph construction process than on adding new features. Although various mapping engines have the same functionality, it is challenging to adopt the various optimization techniques and features of mapping engines. A set of algebraic mapping operators can define the operational semantics of mapping processes, provide a theoretical foundation for mapping languages, and introduce and evaluate a compliant implementation capable of interpreting and executing multiple mapping languages. In this paper, we propose such an algebra based on the SPARQL algebra, bridging the world of knowledge graph construction with query engines. To evaluate that our work is not limited to a single specific mapping language, we translated mapping languages ShExML and RML to our mapping plan composed of algebraic mapping operators. The results of our experimentation to translate and execute all existing test cases and validating their correctness, are ... TODO: add specific results here – the semantics of RML and ShExML are fully captured with our algebraic mapping operators. Furthermore, the mapping plan could also accommodate optimization techniques as a separate process from the mapping process. Algebraic mapping operators decouple mapping engines from the mapping languages, enabling multilingual mapping engines while enjoying the benefits brought by state-of-the-art mapping process optimizations. The proposed set of algebraic mapping operators will lay the foundation for future studies on the theoretical analysis of complexity and expressiveness of mapping languages, and will provide consistency in the execution semantics of mapping engines. Furthermore, the alignment of our algebra with SPARQL will enable further research into heterogeneous data querying.

Keywords: Mapping Algebra, Semantic Web, Mapping Language, Knowledge Graph Construction

1. Introduction

Advances such as optimizations, mapping language features, and interoperability of the mapping engines have been made in the domain of declarative knowledge graph (KG) construction[++]. The mapping engines implementing these techniques infer the operational semantics based on the syntax of the mapping languages used by the engines.

As a result, the mapping engines have different operational semantics from each other when configured to generate the same KG from heterogeneous data. This leads to redundant implementation and incompatibility of features between the different mapping engines.

Due to the lack of theoretical foundations for declarative mapping languages, it is also difficult to prove the correctness of the mapping processes described by the languages. Attempts have been made to formalize the mapping languages[++]. However, the formalization is utilized purely in the context of proving the correctness of an optimization technique. It cannot be generalized to describe the operational semantics of all mapping processes. To

*Corresponding author. E-mail: x.sittminoo@ugent.be.

the best of our knowledge, there is currently no research on the theoretical foundations for declarative mapping languages which could be applied for all declarative mapping languages.

Currently, state-of-the-art research on declarative mapping languages provide no formal description of mapping plans: the exact steps defined formally to construct KG from heterogeneous data. We observe that even if the authors do formally define the mapping plan, it is restricted to a specific declarative mapping language such as RML [1]. One way to provide the theoretical foundation, independent from the mapping languages, is through the definitions of algebraic operators. In this work, we provide the initial set of algebraic operators for mapping algebra to tackle the challenge of providing a theoretical foundation for declarative mapping languages.

The outline of this paper is as follows. We first explore the state-of-the-art techniques for KG construction from heterogeneous data sources using declarative mapping languages (Section 2.1). These mapping languages describe the mapping plan informally and the exact execution semantics is left to the interpretation by the developers of the mapping engines. There is no formal definitions of the mapping plan described by these mapping languages. As inspiration for the formal definitions of the mapping plan, we look to the domain of database querying using relational algebra (Section 2.2). Relational algebra provides the theoretical foundation for all the database systems currently in use. We observe that relational algebra is utilized to formulate the query plan: a plan which describes how a query engine will extract the data from the database and return relevant data to answer the query given by the user. Thus, the definition of a similar set of algebraic operators for *mapping algebra* will enable us to describe mapping plans formally. Similarly, relational algebra has been adapted and also applied in the domain of Semantic Web data querying: also known as SPARQL algebra (Section 2.3). This shows that relational algebra is flexible to be applied in different domains. Since our work lies in the domain of KG construction which is a subdomain of the Semantic Web, we adapt the SPARQL algebra to formulate the *mapping algebra* as defined in Section 5 to show the viability of describing mapping plans formally using algebraic operators.

% TODO: Add more text

2. Related Works

2.1. Mapping Languages

2.2. Relational Algebra

2.3. SPARQL Algebra

3. Methodology

4. Preliminaries

This section introduces terms and definitions adapted from SPARQL, which will be used in the definitions of our mapping algebra. Since this work is inspired by SPARQL algebra, existing definitions and terms by Perez et al. [2] will be reused and slightly adapted. This work will only reintroduce the notations used in SPARQL algebra, readers could refer to the literature for more in-depth definitions [2].

The following pairwise disjoint infinite sets are used in the definition of mapping algebra: V (variables), D (data values), and S (template strings). String template $s \in S$ may contain a set of variables occurring in the template: $var(s)$. In order to *realize* the string template, we introduce *solution mappings* to replace the $vars(s)$ in s.

A *solution mapping* μ is a partial function $\mu : V \rightarrow D$. Provided $s \in S$, $vars(s) \subseteq dom(\mu)$, and $dom(\mu) \subset V$ where μ is defined, $\mu(s)$ is denoted as *realized* string template s with all the $vars(s)$ in s replaced with the values $d \in D$ according to μ.

$$\mu(s) = \{s' | s' = s \text{ where } vars(s) \text{ is replaced with values } d \in range(\mu)\} \quad (1)$$

Although prior work on mapping algebra [3] defined the solution mappings the same way as Perez et al. [2] did, it is worth noting that there is a difference in the definition of the solution mappings in this work. Especially in the type of $range(\mu)$.

This paper extends the prior work on mapping algebra [3] by generalizing $range(\mu)$ type to D, which consists of primitive data types (such as strings, integers, floats, boolean, etc...), instead of restricting it to RDF terms. This will enable future works to utilize the mapping algebra to construct heterogeneous data (such as CSV and XML) and not limit it to the construction of RDF graphs. However, to limit the scope of this work, the focus will be on the generation of RDF graphs using mapping algebras in this paper.

Two mappings μ_1 and μ_2 are *compatible*, if and only if, $\forall v \in dom(\mu_1) \cap dom(\mu_2), \mu_1(v) = \mu_2(v)$; extending this, the union of μ_1 and μ_2, $\mu_1 \cup \mu_2$, is also a solution mapping. Given solution mappings sets Ω_1 and Ω_2, SPARQL algebra [2] defined the *join* ($\bowtie$), and the *union* ($\cup$) between Ω_1 and Ω_2 as follows:

$$\Omega_1 \cup \Omega_2 = \{\mu \mid \mu \in \Omega_2 \text{ or } \mu \in \Omega_2\} \tag{2}$$

$$\Omega_1 \bowtie \Omega_2 = \{\mu_1 \cup \mu_2 \mid \mu_1 \in \Omega_1, \mu_2 \in \Omega_2 \text{ and } \mu_1, \mu_2 \text{ are compatible. }\} \tag{3}$$

When writing sequences of the join and the union operators, parentheses can be avoided since both operators are associative and commutative [2]:

$$\Omega_1 \bowtie \Omega_2 \cup \Omega_3 = \Omega_1 \bowtie (\Omega_2 \cup \Omega_3) \tag{4}$$

5. Definitions

After introducing the preliminary terms and notations, the algebraic mapping operators, and tuples are defined in this section. This paper extends the previous work [3] with four more operators to capture the full semantics of RML, while also improving the definitions of the mapping tuple. We adapt the existing definitions of SPARQL algebra and make a variation of it to derive our algebraic operators for mapping algebra. Throughout this section, examples will be provided by applying these algebraic mapping operators in sequential manner on a small data.

Unlike querying in SPARQL, mapping languages enable users to *fragment* the generated data into different data sinks (e.g. multiple files or web sockets). Thus, we introduce the concept of *fragments*.

Definition 1. A **fragment**, $f \in F$, is grouping of the multiset of solution mappings. It can be seen as a *physical* fragment: a file, a database, or as a *logical* fragment such as specific social context, e.g. information about a person only known by friends. [3]. The set of fragments, F, is also infinite, and pairwise disjoint with the other sets, (V, D, and S) defined in Section 4.

Using the definition of *fragments*, the core data model of the mapping algebra, the *mapping tuple*, is defined as follows.

Definition 2. A **mapping tuple** is a partial multivalued function which maps *fragments* to solutions mappings: $\omega : F \to \Omega$ with Ω, the solution mappings set. A multiset of mapping tuples is ξ.

Definition 3. Two mapping tuples, ω_1 and ω_2, are compatible, if and only if, $\forall f \in dom(\omega_1) \cap dom(\omega_2)$, the associated solution mappings $\omega_1(f) = \mu_1$ and $\omega_2(f) = \mu_2$ are compatible. See Section 4 for solution mappings compatibility.

Utilizing fragments in the mapping tuple enables mapping processes to broadcast solution mappings across multiple downstream operators. Furthermore, it enables the partitioning of the solution mappings during construction based on either user defined conditions or some abstract concept. For example, fragmented mapping tuples could *group* solution mappings according to some social context (e.g. personal information and friend's information)(Table 1).

Table 1

Two mapping tuples describing information related to John. The tuples are fragmented according to *personal* information about John and information about John's *friends*

	Multiset of Solution Mappings			
Fragment	Solution Mapping	?name	?age	?email
$f_{personal}$	μ_1	John Doe	23	john.doe@example.com
$f_{friends}$	μ_2	Susan Sue	25	susan.sue@example.com
$f_{friends}$	μ_3	Alice Joe	26	alice.joe@example.com

5.1. Source Operator

How are the *mapping tuples* generated for further processing by the downstream algebraic operators? In other words, what are the leaf nodes in an algebraic mapping plan?

In an algebraic mapping plan, the *Source* operators are the leaf nodes of the mapping plan and they generate the required mapping tuples for further processing by the downstream algebraic operators.

Definition 4. A **Source** operator generates mapping tuples from heterogeneous data sources. Given a configuration C, a root iterator r, and a set of subiterators I, a source operator generates a multi-set, ξ, of mapping tuples, ω's, where a default fragment f_0 is mapped to a multiset of solution mappings Ω. Configuration C consists of meta-data information about utilizing the data source (e.g. connection ports and credentials for Kafka[1]). Iterators enable querying of the data source to generate individual data records. The combination of root iterator, r, and subiterators, I, enable nested querying of data records from the data source.

$$\begin{aligned} f_0 &= \text{ a default fragment} \\ \Omega &= \{\mu \mid \mu = \text{ flattened data record iterated according to } r \text{ and } I\} \\ \text{Source}(C, r, I) &= \{\omega \mid \omega : f_0 \rightarrow \Omega\} \end{aligned} \tag{5}$$

To clarify the workings of the source operator with root and subiterators, Example 1 shows how a source operator generates mapping tuples from a simple JSON file with nested data.

Listing 1: Example input data in JSON format

```
{
    "peoples": [
        {
            "name" : "John Doe",
            "age"  :        23,
            "email": "john.doe@example.com",
            "pet"  :
            {
                "type": "dog",
                "name": "Bax"
            }
        },
        {
            "name" : "Susan Sue",
            "age"  :         23,
            "email": "susan.sue@example.com"
        }
    ]
}
```

[1] Kafka: https://kafka.apache.org/

Table 2
Representation of the mapping tuple generated by the source operator using data from Listing 1

	Multiset of Solution Mappings					
Fragment	Solution Mapping	?name	?age	?email	?$pet.type	?$pet.name
$f_{default}$	μ_1	John Doe	23	john.doe@example.com	dog	Bax
$f_{default}$	μ_2	Susan Sue	25	susan.sue@example.com		

Example 1. As an example, provided with an input data source such as the JSON file in Listing 1, the source operator could be cofigured with the following C, r, and I:

C: path to the JSON file.
r: $.peoples[*]
I: [$.pet.type, $.pet.name]

The iterators are defined in terms of JSONPath[2] format. The source operator generates a mapping tuple as shown in the Table 2 with a default fragment $f_{default}$. The subiterators are executed relative to the root iterator, and turned into variables using their relative path. This prevents variable collision with existing attributes in the data record. Notice that the orginal attributes in the data record, such as *name* and *age*, are turned into variables by prefixing them with '?'. This is to align the data records with the definition of solution mappings $\mu : V \rightarrow D$ in Section 4.

After the generation of the mapping tuples from heterogeneous data sources by the source operator, these mapping tuples are further processed and transformed by the intermediate algebraic mapping operators. The intermediate algebraic mapping operators are defined as follows: *Projection*, *Fragmenter*, *Join*, *Union*, and *Extend* operators.

5.2. Projection

A standard operation in SPARQL algebra is the *projection* where solution mappings are restricted to a set of attributes provided to the projection operator. This is useful in reducing the number of data that needs to be further processed by the downstream operators. The corresponding *projection* operator in the mapping algebra is defined as follows.

Definition 5. Given a set of variables $P \subseteq V$, a **Projection** operator restricts the variables in the solution mappings μ, associated with the mapping tuple ω, according to P.

$$\begin{aligned} \text{Project}(\mu, P) &= \mu \text{ restricted to variables in } P \\ \text{Project}(\Omega, P) &= \{\text{Project}(\mu, P) \mid \mu \in \Omega\} \\ \text{Project}(\omega, P) &= \{(f, \text{Project}(\Omega, P)) \mid \forall (f, \Omega) \in \omega\} \\ \text{Project}(\xi, P) &= \{\text{Project}(\omega, P) \mid \omega \in \xi\} \end{aligned} \tag{6}$$

Example 2. A projection operator, configured with a set of variables {"?name", "?$pet.name", "?$pet.type"} $\in P$, applied on the generated mapping tuple in Table 2 generates a mapping tuple in Table 3.

5.3. Rename

5.4. Extend

A core operation of the mapping process is the derivation of new values using existing values in the data record. For example, given a data record containing the weight and height of a person, we can calculate the body mass index (BMI) of the person using their weight and height. The following definition of the *extend* operator enables the mapping algebra to derive new values from existing values.

[2]JSONPath: https://goessner.net/articles/JsonPath/

Table 3
Projection operator from Example 2 applied on the mapping tuple shown in Table 2

	Multiset of Solution Mappings			
Fragment	Solution Mapping	?name	?$pet.type	?$pet.name
$f_{default}$	μ_1	John Doe	dog	Bax
$f_{default}$	μ_2	Susan Sue		

Table 4
Extended mapping tuple as described in Example 3

	Multiset of Solution Mappings					
Fragment	Solution Mapping	?name	?$pet.type	?$pet.name	?firstname	?lastname
$f_{default}$	μ_1	John Doe	dog	Bax	John	Doe
$f_{default}$	μ_2	Susan Sue			Susan	Sue

Definition 6. Given a set of pairs $(v_{new}, expr) \in E$, with $v_{new} \notin dom(\mu)$, $v_{new} \in V$ and $expr$ an expression statement. **Extend** operator derives a new value by evaluating the $expr$ expression on the solution mapping, and extends the solution mapping with the generated value, which is coupled to the new variable v_{new}. If evaluating $expr$ causes an error and $v_{new} \notin dom(\mu)$, the extend operator behaves like an identity operator. It is undefined if the variable restriction is violated, which means $v_{new} \in dom(\mu)$. Formally, it is defined as follows.

$$\begin{aligned}
\text{Extend}(\mu, E) &= \mu \cup \{(v_{new}, value) \mid (v_{new}, expr) \in E, v_{new} \notin \text{dom}(\mu) \text{ and value} = \text{expr}(\mu)\} \\
\text{Extend}(\Omega, E) &= \{\text{Extend}(\mu, E) \mid \mu \in \Omega\} \\
\text{Extend}(\omega, E) &= \{(f, \text{Extend}(\Omega, E)) \mid \forall (f, \Omega) \in \omega\} \\
\text{Extend}(\xi, E) &= \{\text{Extend}(\omega, E) \mid \omega \in \xi\}
\end{aligned} \tag{7}$$

Example 3. Provided $\{(?firstname, getFirstName(\_)), (?lastName, getLastName(\_))\} \in E$. The extend operator applied on the mapping tuple shown in Table 3 generates the mapping tuple shown in Table 4.

5.5. Fragmenter

The aforementioned operators do not manipulate the *fragments* part of the mapping tuples but only process the associated solution mappings. To manipulate the fragments of the mapping tuples, the *fragmenter* operator is defined as follows.

Definition 7. Fragmenter operator fragments the mapping tuple into a new fragment f_{new}. Given a partial transformation function, $\delta : F \to F$. A fragmenter operator applies δ on the mapping tuple $\omega = f \to \mu$, and map it into a new mapping tuple $\omega_{mapped} = f_{new} \to \mu$ if $dom(\delta) \subseteq dom(\omega)$ and $f_{new} \in range(\delta)$. When $dom(\delta) \nsubseteq dom(\omega)$, fragmenter operator acts like an identity function. More formally, it is defined as follows.

$$\begin{aligned}
\delta &= \{(f_{old}, f_{new}) \mid f_{old}, f_{new} \in F\} \\
\delta(\omega) &= \{(f, \mu) \mid (f, \mu) \in \omega, f \neq f_{old}\} \cup \{(f_{new}, \mu) \mid (f_{old}, \mu) \in \omega, (f_{old}, f_{new}) \in \delta\} \\
\text{Fragment}(\omega, \delta) &= \begin{cases} \delta(\omega) & \text{if } dom(\delta) \subseteq dom(\omega) \\ \omega & \text{otherwise} \end{cases} \\
\text{Fragment}(\xi, \delta) &= \{\text{Fragment}(\omega, \delta) \mid \omega \in \xi\}
\end{aligned} \tag{8}$$

Table 5
Fragmentation of mapping tuple as described in Example 3

	Multiset of Solution Mappings					
Fragment	Solution Mapping	?name	?$pet.type	?$pet.name	?firstname	?lastname
$f_{contacts}$	μ_1	John Doe	dog	Bax	John	Doe
$f_{contacts}$	μ_2	Susan Sue			Susan	Sue

Example 4. Continuing with the output of the *extend* operator as shown in Table 4. A fragmenter operator with $\delta : \{(f_{default}, f_{contacts})\}$ applied on the mapping tuples generates new mapping tuples shown in Table 5, where the old fragment, $f_{default}$, is mapped to the new fragment, $f_{contacts}$.

5.6. Join

Previously defined operators are unary: they only work on a single multiset of mapping tuples ξ. To combine two multisets of mapping tuples ξ_1, and ξ_2, binary algebraic operators need to be defined. Thus, to support the binary operations, the mapping algebra defines the *natural join*, the *θ-join*, and the *left-join* between ξ_1 and ξ_2 respectively as follows.

Definition 8. Natural join is a binary operator that combines two multisets of mapping tuples ξ_1 and ξ_2 if they are *compatible* (Definition 3). It produces mapping tuples, $\omega_{1\bowtie 2}$, which is a combination of two mapping tuples, $\omega_1 \in \xi_1$ and $\omega_2 \in \xi_2$ that are equal on their fragments and the common variables in the underlying associated solution mappings $\mu_1 \in range(\omega_1), \mu_2 \in range(\omega_2)$. It is formally defined as follows.

$$\begin{aligned}
\text{merge}(\mu_1, \mu_2) &= \{\mu_1 \cup \mu_2 \mid \mu_1 \text{ and } \mu_2 \text{ are compatible}\} \\
\text{Join}(\omega_1, \omega_2) &= \{(f, \text{merge}(\mu_1, \mu_2)) \mid \forall f \in dom(\omega_1) \cap dom(\omega_2), \omega_1(f) = \mu_1, \omega_2(f) = \mu_2\} \\
\text{Join}(\xi_1, \xi_2) &= \{\text{Join}(\omega_1, \omega_2) \mid \exists \omega_1 \in \xi_1, \exists \omega_2 \in \xi_2, \omega_1 \text{ and } \omega_2 \text{ are compatible}\}
\end{aligned} \tag{9}$$

Natural join operator joins mapping tuples if they have common *fragments*, and they also have equal values for all the common *variables* of the associated solution mappings of the fragment. It does not allow user to join mapping tuples based on the values of the different variables of $dom(\mu_1)$ and $dom(\mu_2)$. Furthermore, natural join only checks for the *equality* condition on the common variables and fragments: it does not use other predicate functions such as $\leqslant$ or $\geqslant$. *Theta-join*, a more general form of the natural join, enables the use of a predicate function, θ, on specified variables and fragments to join two multisets of mapping tuples. It is defined as follows.

Definition 9. Given a binary predicate function, ${}_{v_1}\theta_{v_1} : \Omega \times \Omega \rightarrow boolean$, with variables v_1, and v_2, where $v_1 \in dom(\mu_1), v_2 \in dom(\mu_2)$, and two multisets of mapping tuples ξ_1 and ξ_2. **Theta-join** combines two multisets of mapping tuples ξ_1, and ξ_2, if for the same fragment $f \in dom(\omega_1) \cap dom(\omega_2)$, ${}_{v_1}\theta_{v_2}(\mu_1, \mu_2)$ evaluates to *true* for solution mappings $\mu_1 = \omega_1(f)$ and $\mu_2 = \omega_2(f)$. More formally, it is defined as follows.

$$\begin{aligned}
{}_{v_1}\theta_{v_2}(\mu_1, \mu_2) &= \begin{cases} true, & \text{if } \mu_1(v_1) = \mu_2(v_2), v_1 \in dom(\mu_1), v_2 \in dom(\mu_2) \\ false, & \text{otherwise} \end{cases} \\
\text{ThetaJoin}(\mu_1, \mu_2, {}_{v_1}\theta_{v_2}) &= \{\mu_1 \cup \mu_2 \mid {}_{v_1}\theta_{v_2}(\mu_1, \mu_2) \text{ evaluates to true }\} \\
\text{ThetaJoin}(\omega_1, \omega_2, {}_{v_1}\theta_{v_2}) &= \{(f, \text{ThetaJoin}(\mu_1, \mu_2, {}_{v_1}\theta_{v_2})) \mid \forall f \in dom(\omega_1) \cap dom(\omega_2), \omega_1(f) = \mu_1, \omega_2(f) = \mu_2\} \\
\text{ThetaJoin}(\xi_1, \xi_2, {}_{v_1}\theta_{v_2}) &= \{\text{ThetaJoin}(\omega_1, \omega_2, {}_{v_1}\theta_{v_2}) \mid \omega_1 \in \xi_1, \omega_2 \in \xi_2\}
\end{aligned} \tag{10}$$

Table 6
Mapping tuples generated from another data source about pets

	Multiset of Solution Mappings			
Fragment	Solution Mapping	?$type	?$pet.name	?pet.age
$f_{contacts}$	μ_{a1}	dog	Bax	10
$f_{contacts}$	μ_{a2}	cat	Coco	3
$f_{contacts}$	μ_{a3}	dog	Max	5

Table 7
Mapping tuples generated as the output of applying natural join operator on Table 5 and Table 6 as ξ_1 and ξ_2 respectively.

	Multiset of Solution Mappings							
Fragment	Solution Mapping	?name	?$pet.type	?$pet.name	?firstname	?lastname	?pet.age	?type
$f_{contacts}$	$\mu_1 \cup \mu_{a1}$	John Doe	dog	Bax	John	Doe	10	dog

Natural and theta-join filters out solution mappings which do not satisfy the predicate function. The filtering of solution mappings results in the loss of information which might require extra processing steps to retain them.

Thus, a new binary algebraic operator, which retains the solution mappings that do not satisfy the given predicate function, needs to be defined. SPARQL and relational algebra have definitions for such a group of binary operators called *outer-joins*. *Left outer-join* operator is a binary operator that retains the solution mappings from the *left* multisets of mapping tuples even if the given solution mappings do not satisfy the predicate function. Other outer-join operators, such as *right outer-join*, and *full outer-join* operators, can be derived from the left outer-join operator.

Definition 10. Given two multisets of mapping tuples, ξ_1 and ξ_2, a predicate function, ${}_{v_1}\theta_{v_2}$. If fragment $f \in dom(\omega_1) \cap dom(\omega_2)$, **Left outer-join** combines two multisets of mapping tuples, ξ_1 and ξ_2 based on the *boolean condition* after evaluating ${}_{v_1}\theta_{v_2}(\mu_1, \mu_2)$ as follows. If it is true, it behaves the same as the *theta-join* operator. Otherwise, μ_1 is merged with μ_2^{null} where $\mu_2^{null} = \{(v, null) \mid \forall v \in dom(\mu_2)\}$. More formally, it is defined as follows.

$$
\begin{aligned}
\mu_2^{null} &= \{(v, null) \mid \forall v \in dom(\mu_2)\} \\
\text{LeftJoin}(\mu_1, \mu_2, {}_{v_1}\theta_{v_2}) &= \begin{cases} \mu_1 \cup \mu_2, & \text{if } {}_{v_1}\theta_{v_2}\ (\mu_1, \mu_2) \text{ is } \textit{true} \\ \mu_1 \cup \mu_2^{null}, & \text{otherwise} \end{cases} \\
\text{LeftJoin}(\omega_1, \omega_2, {}_{v_1}\theta_{v_2}) &= \{(f, \text{LeftJoin}(\mu_1, \mu_2, {}_{v_1}\theta_{v_2})) \mid \forall f \in dom(\omega_1) \cap dom(\omega_2), \omega_1(f) = \mu_1, \omega_2(f) = \mu_2\ \} \\
\text{LeftJoin}(\xi_1, \xi_2, {}_{v_1}\theta_{v_2}) &= \{\text{LeftJoin}(\omega_1, \omega_2, {}_{v_1}\theta_{v_2}) \mid \omega_1 \in \xi_1, \omega_2 \in \xi_2\}
\end{aligned}
\tag{11}
$$

We provide the following examples, where the three aforementioned join operators are applied on the mapping tuples shown in Table 5 and Table 6 as ξ_1 and ξ_2 respectively.

Example 5. Applying the natural join operator on the mapping tuples form Table 5, will produce the output as shown in Table 7. The natural join creates a union of solution mappings based on the value of the common variables. In the example, the common variable between the two different multisets of solution mappings is *?pet.name*. Since only μ_1 and μ_{a1} have the same value for the variable *?pet.name*, the output only contains $\mu_1 \cup \mu_{a1}$ and the other solution mappings are dropped.

Example 6. Provided with the predicate function, ${}_{?pet.type}\theta_{?type}$, for equality check on the variables *?pet.type* and *?type*. Applying Theta-join operator on Table 5 and Table 6, as ξ_1 and ξ_2 respectively, produces the output in Table 8

Table 8
Output of theta-join operator as described in Example 6

	Multiset of Solution Mappings								
Fragment	Solution Mapping	?name	?$pet.type	?$pet.name	?firstname	?lastname	?pet.age	?type	?name
$f_{contacts}$	$\mu_1 \cup \mu_{a1}$	John Doe	dog	Bax	John	Doe	10	dog	
$f_{contacts}$	$\mu_1 \cup \mu_{a3}$	John Doe	dog	Max	John	Doe	10	dog	

5.7. Serializer

5.8. Target

6. Implementation

6.1. RML translation

6.1.1. Algorithm

6.2. ShExML translation

6.2.1. Algorithm

6.3. Algebraic mapping engine

7. Evaluation

7.1. Generated Mapping Plan Comparison

Comparison of the mapping plan generated from RML against ShExML.

- Number of nodes
- Number of edges (fragmentation)
- Type of nodes

7.2. Initial Mapping Plan Optimization

Simple mapping plan rewriting optimization

1. Generate a really naive mapping plan from RML
2. Rewrite the naive mapping plan to be more 'optimal' using rewriting rules from relational algebras
3. Evaluate the mapping engine's **memory**, and **execution time** on naive and 'optimized' plan

7.3. Performance comparison against existing engines

Compare the performance of algebraic mapping engine against the **reference implementations** of **RML**, and **ShExML**.

Metrics to be measured:

1. Memory usage
2. Execution time

8. Results

9. Conclusion

10. Future Works

10.1. Algebra formalization

10.2. Mapping Plan Optimizations

10.3. Performant generic mapping engine

References

[1] E. Iglesias, S. Jozashoori and M.-E. Vidal, Scaling up knowledge graph creation to large and heterogeneous data sources, *Journal of Web Semantics* **75** (2023). doi:10.1016/j.websem.2022.100755. http://arxiv.org/abs/2201.09694.

[2] J. Pérez, M. Arenas and C. Gutierrez, Semantics and Complexity of SPARQL, *ACM Trans. Database Syst.* **34**(3) (2009). doi:10.1145/1567274.1567278.

[3] Min Oo, Sitt and De Meester, Ben and Taelman, Ruben and Colpaert, Pieter, Towards algebraic mapping operators for knowledge graph construction, 2023, p. 5. ISBN 978-3-031-47239-8.

[4] B. De Meester, T. Seymoens, A. Dimou and R. Verborgh, Implementation-independent function reuse, *Future Generation Computer Systems* **110** (2020), 946–959. doi:https://doi.org/10.1016/j.future.2019.10.006. https://www.sciencedirect.com/science/article/pii/S0167739X19303723.